Name:- Raghav Anil Bhise
Roll No. :- 391061
PRN:- 22311792

1. Title

Implementation of Stemming and Lemmatization in Natural Language Processing

---

2. Aim

To study and implement **stemming and lemmatization** techniques for text normalization in Natural Language Processing.

---

3. Objectives

- To understand the need for text normalization in NLP
- To implement **stemming** using stemming algorithms
- To implement **lemmatization** using lexical analysis
- To compare the outputs of stemming and lemmatization

---

4. Theory

4.1 Text Normalization in NLP

Text normalization is the process of converting words into a standard or base form. It helps reduce variations of the same word and improves the performance of NLP models.

4.2 Stemming

Stemming is the process of reducing words to their **root or base form** by removing suffixes.

- It does not consider grammar or meaning

- The resulting word may not be a valid dictionary word

Example:

- Playing → Play
- Studies → Studi

4.3 Lemmatization

Lemmatization converts words into their **meaningful base form (lemma)** using vocabulary and grammar rules.

- Produces valid dictionary words
- More accurate but computationally expensive

Example:

- Playing → Play
- Studies → Study

4.4 Difference Between Stemming and Lemmatization

Feature	Stemming	Lemmatization
Approach	Rule-based	Dictionary-based
Output	May not be meaningful	Meaningful words
Accuracy	Lower	Higher
Speed	Faster	Slower

---

5. Program Codes

Porter's Stemmer

It uses a set of heuristic rules to iteratively remove suffixes. Example: EED -> EE means "if the word has at least one vowel and consonant plus EED ending, change the ending to EE" as 'agreed' becomes 'agree'.

[1] 3s
```
from nltk.stem import PorterStemmer,LancasterStemmer, RegexpStemmer, SnowballStemmer
```

[2] 0s
```
l = LancasterStemmer()
r = RegexpStemmer("ing")
p = PorterStemmer()
s = SnowballStemmer('english')
```

[3] 0s
```
s.stem("bathing")
```

```
'bath'
```

+ Code + Text

[4] 0s
```
from nltk.stem import SnowballStemmer
SnowballStemmer.languages
```

```
('arabic',
 'danish',
 'dutch',
 'english',
 'finnish',
 'french',
 'german',
 'hungarian',
 'italian',
 'norwegian',
 'porter',
 'portuguese',
 'romanian',
 'russian',
 'spanish',
 'swedish')
```

[5] 0s
```
from nltk.stem import PorterStemmer

# Create a Porter Stemmer instance
porter_stemmer = PorterStemmer()

# Example words for stemming
words = ["running", "jumps", "happily", "running", "happilly"]

# Apply stemming to each word
stemmed_words = [porter_stemmer.stem(word) for word in words]

# Print the results
print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

```
Original words: ['running', 'jumps', 'happily', 'running', 'happilly']
Stemmed words: ['run', 'jump', 'happili', 'run', 'happilli']
```

[6] 0s
```
from random import sample
from nltk.stem.porter import PorterStemmer

ps = PorterStemmer()
```

Lancaster Stemmer

The Lancaster stemmers are more aggressive and dynamic compared to the other two stemmers. The stemmer is really faster, but the algorithm is really confusing when dealing with small words. But they are not as efficient as Snowball Stemmers. The Lancaster stemmers save the rules externally and basically uses an iterative algorithm.

+ Code + Text

[9] 0s
```
from nltk.stem import LancasterStemmer

# Create a Lancaster Stemmer instance
stemmer = LancasterStemmer()

# Example words to stem
words_to_stem = ['running', 'jumped', 'happily', 'quickly', 'foxes']

# Apply Lancaster Stemmer
stemmed_words = [stemmer.stem(word) for word in words_to_stem]

# Print the results
print("Original words:", words_to_stem)
print("Stemmed words:", stemmed_words)
```

```
Original words: ['running', 'jumped', 'happily', 'quickly', 'foxes']
Stemmed words: ['run', 'jump', 'happy', 'quick', 'fox']
```

Lancaster Stemmer

[12]
0s

```python
import nltk
from nltk.stem import LancasterStemmer
from nltk.tokenize import word_tokenize

nltk.download('punkt_tab') # Download the necessary 'punkt_tab' resource

# Initialize the Porter Stemmer
stemmer = LancasterStemmer()

# Example sentence
sentence = "Love looks not with the eyes but with the mind, and therefore is winged Cupid painted blind."

# Tokenize the sentence
words = word_tokenize(sentence)

# Stemming the words
stemmed_words = [stemmer.stem(word) for word in words]

# Print the stemmed words
print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

```
tk_data] Downloading package punkt_tab to /root/nltk_data...
tk_data]   Unzipping tokenizers/punkt_tab.zip.
ginal words: ['Love', 'looks', 'not', 'with', 'the', 'eyes', 'but', 'with', 'the', 'mind', ',', 'and', 'therefore', 'is', 'winged', 'Cupid', 'painted', 'blind', '
emmed words: ['lov', 'look', 'not', 'with', 'the', 'ey', 'but', 'with', 'the', 'mind', ',', 'and', 'theref', 'is', 'wing', 'cupid', 'paint', 'blind', '.']
```

Regexp Stemmer

The Regexp Stemmer, or Regular Expression Stemmer, is a stemming algorithm that utilizes regular expressions to identify and remove suffixes from words. It allows users to define custom rules for stemming by specifying patterns to match and remove.

[13]
0s

```python
from nltk.stem import RegexpStemmer

# Create a Regexp Stemmer with a custom rule
custom_rule = r'ing$'
regexp_stemmer = RegexpStemmer(custom_rule)

# Apply the stemmer to a word
word = 'running'
stemmed_word = regexp_stemmer.stem(word)

print(f'Original Word: {word}')
print(f'Stemmed Word: {stemmed_word}')
```

```
Original Word: running
Stemmed Word: runn
```

Snowball Stemmer

An extension of the Porter Stemmer with more robust rules. The Snowball Stemmer, compared to the Porter Stemmer, is multi-lingual as it can handle non-English words. It supports various languages and is based on the 'Snowball' programming language, known for efficient processing of small strings.

[8]
0s

```python
from nltk.stem import SnowballStemmer

# Choose a language for stemming, for example, English
stemmer = SnowballStemmer(language='english')

# Example words to stem
words_to_stem = ['running', 'jumped', 'happily', 'quickly', 'foxes']

# Apply Snowball Stemmer
stemmed_words = [stemmer.stem(word) for word in words_to_stem]

# Print the results
print("Original words:", words_to_stem)
print("Stemmed words:", stemmed_words)
```

```
Original words: ['running', 'jumped', 'happily', 'quickly', 'foxes']
Stemmed words: ['run', 'jump', 'happili', 'quick', 'fox']
```

Lemmatization

Lemmatization is the process of reducing words to their base or dictionary form, known as the lemma. This technique considers the context and the meaning of the words, ensuring that the base form belongs to the language's dictionary. For example, the words "running," "ran," and "runs" are all lemmatized to the lemma "run."

[14]

```
import nltk
nltk.download('wordnet')
from nltk.stem import WordNetLemmatizer
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

[15]

```
wl = WordNetLemmatizer()
```

+ Code + Text

[16]

```
wl.lemmatize("mice")
```

```
'mouse'
```

[17]

```
import nltk
nltk.download('wordnet')
from nltk.stem import WordNetLemmatizer

# Create WordNetLemmatizer object
wnl = WordNetLemmatizer()

# single word lemmatization examples
list1 = ['kites', 'babies', 'dogs', 'flying', 'smiling',
         'driving', 'died', 'tried', 'feet']
for words in list1:
    print(words + " ---> " + wnl.lemmatize(words))
```

```
kites ---> kite
babies ---> baby
dogs ---> dog
flying ---> flying
smiling ---> smiling
driving ---> driving
died ---> died
tried ---> tried
feet ---> foot
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Sentence lemmatization examples

[18]

```
import nltk
nltk.download('punkt_tab')
from nltk.stem import WordNetLemmatizer
string = 'the cat is sitting with the bats on the striped mat under many flying geese'
wnl = WordNetLemmatizer()

# Converting String into tokens
list2 = nltk.word_tokenize(string)
print(list2)
#> ['the', 'cat', 'is', 'sitting', 'with', 'the', 'bats', 'on',
# 'the', 'striped', 'mat', 'under', 'many', 'flying', 'geese']

lemmatized_string = ' '.join([wnl.lemmatize(words) for words in list2])

print(lemmatized_string)
#> the cat is sitting with the bat on the striped mat under many flying goose
```

```
['the', 'cat', 'is', 'sitting', 'with', 'the', 'bats', 'on', 'the', 'striped', 'mat', 'under', 'many', 'flying', 'geese']
the cat is sitting with the bat on the striped mat under many flying goose
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

After stopwords removal

[19]

```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

[20]

```
from nltk.corpus import stopwords
for words in list2:
  if words not in stopwords.words('english'):
    print(words + " ---> " + wnl.lemmatize(words))
```

```
cat ---> cat
sitting ---> sitting
bats ---> bat
striped ---> striped
mat ---> mat
many ---> many
flying ---> flying
geese ---> goose
```

Wordnet Lemmatizer (with POS tag)

Words like 'sitting', 'flying' etc remained the same after lemmatization. This is because these words are treated as a noun in the given sentence rather than a verb. To overcome come this, we use POS (Part of Speech) tags.

[21]
0s

```python
# WORDNET LEMMATIZER (with appropriate pos tags)

import nltk
from nltk.stem import WordNetLemmatizer
nltk.download('averaged_perceptron_tagger_eng')
from nltk.corpus import wordnet

lemmatizer = WordNetLemmatizer()

# Define function to lemmatize each word with its POS tag

# POS_TAGGER_FUNCTION : TYPE 1
def pos_tagger(nltk_tag):
  if nltk_tag.startswith('J'):
    return wordnet.ADJ
  elif nltk_tag.startswith('V'):
    return wordnet.VERB
  elif nltk_tag.startswith('N'):
    return wordnet.NOUN
  elif nltk_tag.startswith('R'):
    return wordnet.ADV
  else:
    return None

sentence = 'the cat is sitting with the bats on the striped mat under many badly flying geese'

# tokenize the sentence and find the POS tag for each token
```

[21]
0s

```python
# tokenize the sentence and find the POS tag for each token
pos_tagged = nltk.pos_tag(nltk.word_tokenize(sentence))

print(pos_tagged)
#>[('the', 'DT'), ('cat', 'NN'), ('is', 'VBZ'), ('sitting', 'VBG'), ('with', 'IN'),
# ('the', 'DT'), ('bats', 'NNS'), ('on', 'IN'), ('the', 'DT'), ('striped', 'JJ'),
# ('mat', 'NN'), ('under', 'IN'), ('many', 'JJ'), ('flying', 'VBG'), ('geese', 'JJ')]

# As you may have noticed, the above pos tags are a little confusing.

# we use our own pos_tagger function to make things simpler to understand.
wordnet_tagged = list(map(lambda x: (x[0], pos_tagger(x[1])), pos_tagged))
print(wordnet_tagged)
#>[('the', None), ('cat', 'n'), ('is', 'v'), ('sitting', 'v'), ('with', None),
# ('the', None), ('bats', 'n'), ('on', None), ('the', None), ('striped', 'a'),
# ('mat', 'n'), ('under', None), ('many', 'a'), ('flying', 'v'), ('geese', 'a')]

lemmatized_sentence = []
for word, tag in wordnet_tagged:
  if tag is None:
    # if there is no available tag, append the token as is
    lemmatized_sentence.append(word)
  else:
    # else use the tag to lemmatize the token
    lemmatized_sentence.append(lemmatizer.lemmatize(word, tag))
lemmatized_sentence = " ".join(lemmatized_sentence)

print(lemmatized_sentence)
#> the cat can be sit with the bat on the striped mat under many fly geese
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[('the', 'DT'), ('cat', 'NN'), ('is', 'VBZ'), ('sitting', 'VBG'), ('with', 'IN'), ('the', 'DT'), ('bats', 'NNS'), ('on', 'IN'), ('the', 'DT'), ('striped', 'JJ'),
[('the', None), ('cat', 'n'), ('is', 'v'), ('sitting', 'v'), ('with', None), ('the', None), ('bats', 'n'), ('on', None), ('the', None), ('striped', 'a'), ('mat',
the cat be sit with the bat on the striped mat under many badly fly geese
```

TextBlob

TextBlob is a python library used for processing textual data. It provides a simple API to access its methods and perform basic NLP tasks.

Download TextBlob package : In your anaconda prompt or terminal, type: pip install textblob

[22]
0s

```python
from textblob import TextBlob, Word

my_word = 'cats'

# create a Word object
w = Word(my_word)

print(w.lemmatize())
#> cat

sentence = 'the bats saw the cats with stripes hanging upside down by their feet.'

s = TextBlob(sentence)
lemmatized_sentence = " ".join([w.lemmatize() for w in s.words])

print(lemmatized_sentence)
#> the bat saw the cat with stripe hanging upside down by their foot
```

```
cat
the bat saw the cat with stripe hanging upside down by their foot
```

TextBlob (with POS tag)

Same as in Wordnet approach without using appropriate POS tags, we observe the same limitations in this approach as well. So, we use one of the more powerful aspects of the TextBlob module the 'Part of Speech' tagging to overcome this problem.

[23]

```python
from textblob import TextBlob

# Define function to lemmatize each word with its POS tag

# POS_TAGGER_FUNCTION : TYPE 2
def pos_tagger(sentence):
  sent = TextBlob(sentence)
  tag_dict = {"J": 'a', "N": 'n', "V": 'v', "R": 'r'}
  words_tags = [(w, tag_dict.get(pos[0], 'n')) for w, pos in sent.tags]
  lemma_list = [wd.lemmatize(tag) for wd, tag in words_tags]
  return lemma_list

# Lemmatize
sentence = "the bats saw the cats with stripes hanging upside down by their feet"
lemma_list = pos_tagger(sentence)
lemmatized_sentence = " ".join(lemma_list)
print(lemmatized_sentence)
#> the bat saw the cat with stripe hang upside down by their foot
t_blob = TextBlob(sentence)
lemmatized_sentence = " ".join([w.lemmatize() for w in t_blob.words])
print(lemmatized_sentence)
#> the bat saw the cat with stripe hanging upside down by their foot
```

```
the bat saw the cat with stripe hang upside down by their foot
the bat saw the cat with stripe hanging upside down by their foot
```

Natural Language Processing with Tokenization and Lemmatization

[24]

```python
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import wordnet

# Download necessary NLTK data
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
True
```

[25]

```python
# Sample text
text = "The striped bats are hanging on their feet for best"

# Tokenize the text
words = nltk.word_tokenize(text)

# Initialize the stemmer and lemmatizer
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

# Apply stemming
stemmed_words = [stemmer.stem(word) for word in words]

# Function to get the part of speech tag for lemmatization
def get_wordnet_pos(word):
    tag = nltk.pos_tag([word])[0][1][0].upper()
    tag_dict = {"J": wordnet.ADJ, "N": wordnet.NOUN, "V": wordnet.VERB, "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)

# Apply lemmatization
lemmatized_words = [lemmatizer.lemmatize(word, get_wordnet_pos(word)) for word in words]

# Print results
print("Original Text: ", text)
print("Tokenized Words: ", words)
print("Stemmed Words: ", stemmed_words)
print("Lemmatized Words: ", lemmatized_words)
```

```
Original Text:  The striped bats are hanging on their feet for best
Tokenized Words:  ['The', 'striped', 'bats', 'are', 'hanging', 'on', 'their', 'feet', 'for', 'best']
Stemmed Words:  ['the', 'stripe', 'bat', 'are', 'hang', 'on', 'their', 'feet', 'for', 'best']
Lemmatized Words:  ['The', 'strip', 'bat', 'be', 'hang', 'on', 'their', 'foot', 'for', 'best']
```

6. Result and Outcomes

Result

- The input text was tokenized successfully.
- Words were reduced to their root form using stemming.
- Words were converted into meaningful base forms using lemmatization.

Outcomes

- Understood the concept of text normalization
- Learned the difference between stemming and lemmatization
- Observed how lemmatization provides more accurate results
- Gained practical experience using NLTK stemmer and lemmatizer

---

7. Conclusion

Stemming and lemmatization are important text preprocessing techniques in Natural Language Processing. While stemming provides a faster but less accurate approach, lemmatization ensures grammatical correctness and better semantic understanding. Through this experiment, both techniques were implemented and compared successfully, highlighting their roles in effective NLP preprocessing.